Raport de Testare a Securitatii

Proiect #1 - Hack your own app before someone else does

Student: Lupes Ioan-Marian

Curs: Dezvoltarea Aplicatiilor Software Securizate

1. Introducere

In cadrul acestui proiect am construit o aplicatie web numita Deskly, o platforma de tip SaaS pentru managementul incidentelor IT. Aplicatia a fost construita intentionat cu vulnerabilitati de securitate tipice, dupa care am jucat rolul de pentester pentru a le demonstra si de developer pentru a le repara.

Ideea principala era sa vad cum arata un ciclu complet de securitate: construiesti ceva, il spargi, il repari. Mi s-a parut destul de interesant pentru ca nu ma gandeam inainte cat de usor pot fi exploatate vulnerabilitatile daca nu esti atent la detalii.

1.1 Descrierea aplicatiei

Deskly este o aplicatie web cu doua tipuri de utilizatori:

- Analyst - poate crea si gestiona propriile tichete de incident
- Manager - poate vedea toate tichetele echipei si are drepturi administrative

Functionalitatile principale implementate:

- Autentificare cu register / login / logout
- CRUD complet pentru tichete (creare, vizualizare, editare, stergere)
- Sistem de roluri cu control al accesului
- Cautare si filtrare tichete
- Audit log vizibil doar pentru Manager

1.2 Tehnologii folosite

Componenta	Tehnologie
Backend	Python 3.12 + Flask 3.0
Baza de date	SQLite + SQLAlchemy ORM
Autentificare	Flask-Login + bcrypt
Protectie CSRF	Flask-WTF (versiunea securizata)
Sanitizare input	bleach 6.1
Mediu de lucru	Ubuntu 24.04 LTS in VirtualBox
Testing	pytest + 17 teste automate

2. Setup mediu de lucru

Am rulat totul intr-o masina virtuala Ubuntu 24.04 LTS in VirtualBox pe Windows 11. Am ales Ubuntu pentru ca e mai usor de configurat Python si instrumentele de securitate pe Linux decat pe Windows.

2.1 Instalare dependente

Dupa ce am pornit VM-ul, am instalat dependentele necesare:

```
sudo apt update && sudo apt install python3-venv python3-pip sqlite3 -y
```

Am dezarhivat proiectul pe Desktop si am creat doua medii virtuale separate pentru versiunea vulnerabila si cea securizata:

```
cd ~/Desktop/deskly/vulnerable
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

2.2 Structura proiectului

Proiectul are doua versiuni clare, conform cerintei:

- vulnerable/ - versiunea initiala MVP cu vulnerabilitatile introduse intentionat
- fixed/ - versiunea securizata cu toate fix-urile aplicate
- tests/ - 17 teste automate de securitate
- poc/ - fisiere pentru demonstrarea atacurilor (CSRF PoC HTML, comenzi curl)

Hostname-ul VM-ului: ionut-lupes (vizibil in toate capturile de terminal)

3. Implementare MVP

Am implementat toate functionalitatile obligatorii din cerinta. Mai jos descriu pe scurt cum am construit fiecare modul.

3.1 Autentificare

Fluxul de autentificare include register, login si logout. In versiunea vulnerabila am lasat intentionat mai multe probleme: parola stocata in plain text, fara rate limiting, mesaje de eroare care dezvaluie informatii despre utilizatori.

In versiunea securizata am remediat toate acestea: bcrypt cu cost factor 12, lockout dupa 5 incercari gresite, mesaje generice de eroare.

3.2 Sistemul de tichete (CRUD)

Aplicatia permite crearea, vizualizarea, editarea si stergerea de tichete. Fiecare tichet are: titlu, descriere, severitate (LOW/MED/HIGH), status (OPEN/IN_PROGRESS/RESOLVED) si owner_id.

In versiunea vulnerabila ID-urile tichetelor sunt numere intregi secventiale (1, 2, 3...) ceea ce face atacul IDOR trivial. In versiunea securizata am folosit UUID-uri aleatorii.

3.3 Cautare si filtrare

Am implementat cautare dupa titlu si descriere. In versiunea vulnerabila query-ul era construit prin concatenare de string-uri, ceea ce permite SQL Injection. In versiunea securizata am folosit ORM parametrizat prin SQLAlchemy.

3.4 Audit Log

Toate actiunile importante sunt inregistrate automat: login, logout, creare/editare/stergere tichete, schimbare status, tentative de acces neautorizat. Audit log-ul este vizibil doar pentru utilizatorii cu rol Manager.

4. Prezentare vulnerabilitati (OWASP Mapping)

Am identificat si implementat intentionat 6 categorii de vulnerabilitati, conform cerintei proiectului. Toate corespund categoriilor din OWASP Top 10.

#	Categorie OWASP	Vulnerabilitate introdusa	Impact potential
1	Broken Access Control (IDOR)	GET /tickets/{id} fara verificare owner	Acces la date confidentiale ale altor utilizatori
2	Injection (SQLi)	Concatenare string in query search	Exfiltrare date, bypass autentificare
3	XSS Stored	Descriere ticket afisata cu |safe	Furt cookie sesiune, session hijacking
4A	CSRF	Endpoint status fara token CSRF	Actiuni neautorizate in numele userului
4B	Session Management	Cookie fara HttpOnly/Secure/SameSite	Furt cookie din JavaScript
5	Info Disclosure	Stack trace afisat clientului	Dezvaluire structura interna, cai fisiere
6	Password Storage	Parole stocate in plain text	Compromiterea tuturor conturilor la breach DB

5. Demonstrarea atacurilor (Proof of Concept)

5.1 PoC #1 - IDOR (Broken Access Control)

Descriere tehnica:

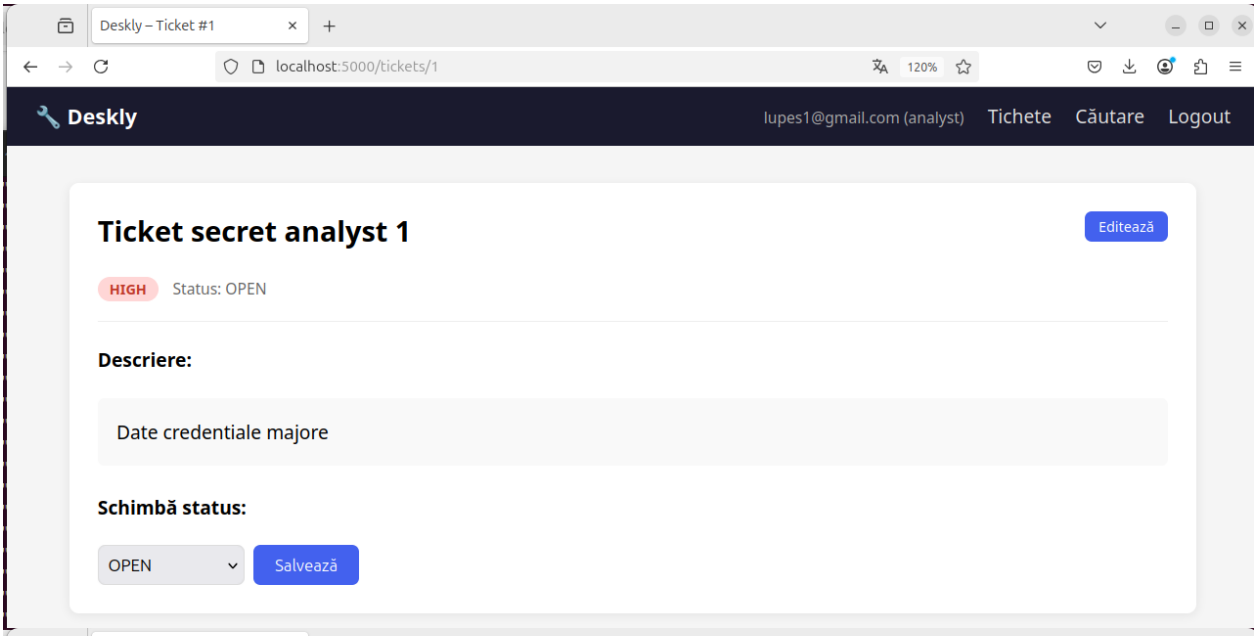
Endpoint-ul GET /tickets/{id} nu verifica daca utilizatorul autentificat este proprietarul tichetului sau are rol de manager. Orice utilizator autentificat poate accesa orice tichet modificand ID-ul din URL.

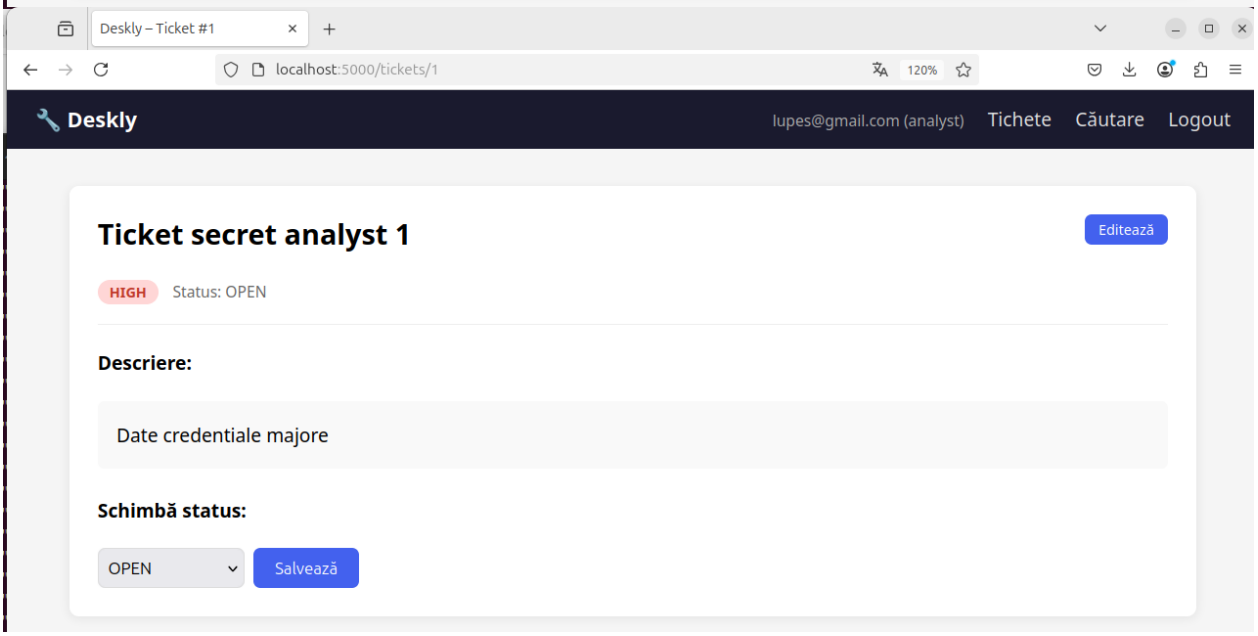
Codul vulnerabil:

```
@tickets_bp.route('/tickets/<int:ticket_id>')
@login_required
def view_ticket(ticket_id):
    ticket = Ticket.query.get_or_404(ticket_id)
    # LIPSA: verificare ownership/rol
    return render_template('tickets/view.html', ticket=ticket)
```

Pasi atac:

1. M-am autentificat cu contul analyst2 (lupes1@gmail.com)
2. Am creat un tichet cu datele proprii (ticket #2)
3. Am accesat manual URL-ul: http://localhost:5000/tickets/1
4. Aplicatia a returnat tichetul lui analyst1, desi eram logat ca analyst2





Impact:

Un atacator poate accesa, modifica sau sterge tichetele tuturor utilizatorilor din sistem, inclusiv informatii confidentiale despre incidente de securitate.

5.2 PoC #2 - SQL Injection

Descriere tehnica:

Functia de cautare construieste query-ul SQL prin concatenare directa a input-ului utilizatorului. Acest lucru permite unui atacator sa modifice logica query-ului sau sa extraga date din alte tabele.

Codul vulnerabil:

```
sql = f"SELECT * FROM tickets WHERE title LIKE '%{query}%'"
               OR description LIKE '%{query}%'"
result = db.session.execute(db.text(sql))
```

Payload #1 - OR 1=1 (returneaza toate tichetele):

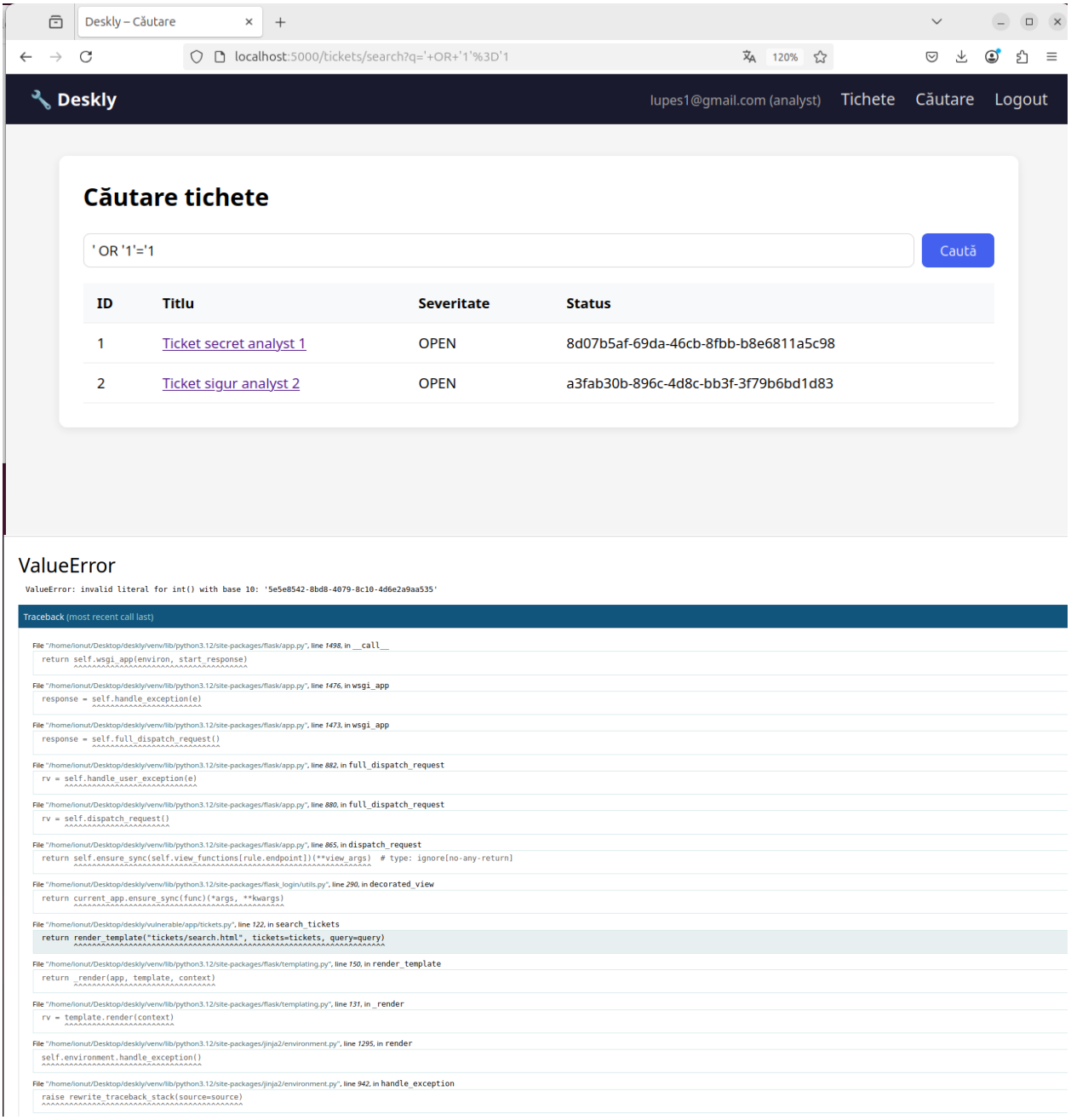
```
' OR '1'='1
```

Rezultat: Au aparut tichetele tuturor utilizatorilor, nu doar ale celui autentificat.

Payload #2 - UNION SELECT (extrage erori verbose):

```
' UNION SELECT id,email,password,email,email,datetime('now'),datetime('now')
FROM users --
```

Rezultat: Stack trace complet cu query-ul SQL expus, path-uri interne ale serverului, versiunea framework-ului. Asta demonstreaza si vulnerabilitatea #5 (Info Disclosure) simultan.



Impact:

Atacatorul poate extrage date din orice tabel al bazei de date, inclusiv parole, sau poate modifica/sterge date.

5.3 PoC #3 - XSS Stored

Descriere tehnica:

Descrierea tichetului este afisata in template cu filtrul |safe din Jinja2, care dezactiveaza escaping-ul HTML. Orice cod JavaScript injectat in descriere se executa in browserul oricarui utilizator care vizualizeaza tichetul.

Codul vulnerabil din template:

```
{{ ticket.description | safe }}
```

Payload folosit:

```
<script>alert('XSS - cookie: ' + document.cookie)</script>
```

Rezultat: La deschiderea tichetului a aparut un popup cu cookie-ul de sesiune complet al utilizatorului autentificat. Acest cookie poate fi folosit de un atacator pentru session hijacking - adica sa se logheze in contul victimei fara a cunoaste parola.

Impact:

Un atacator poate fura sesiunile tuturor utilizatorilor care vizualizeaza tichetul compromis, inclusiv ale managerilor cu drepturi administrative.

5.4 PoC #4A - CSRF

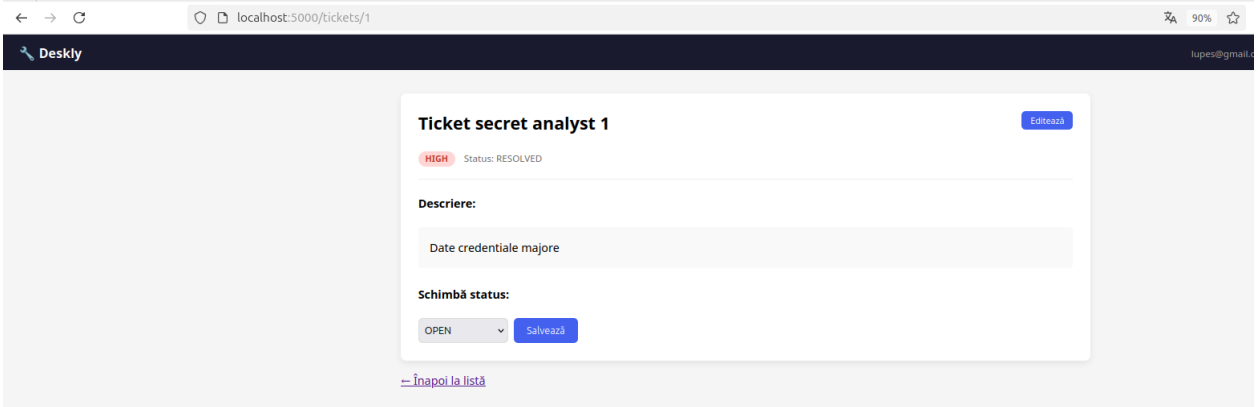
Descriere tehnica:

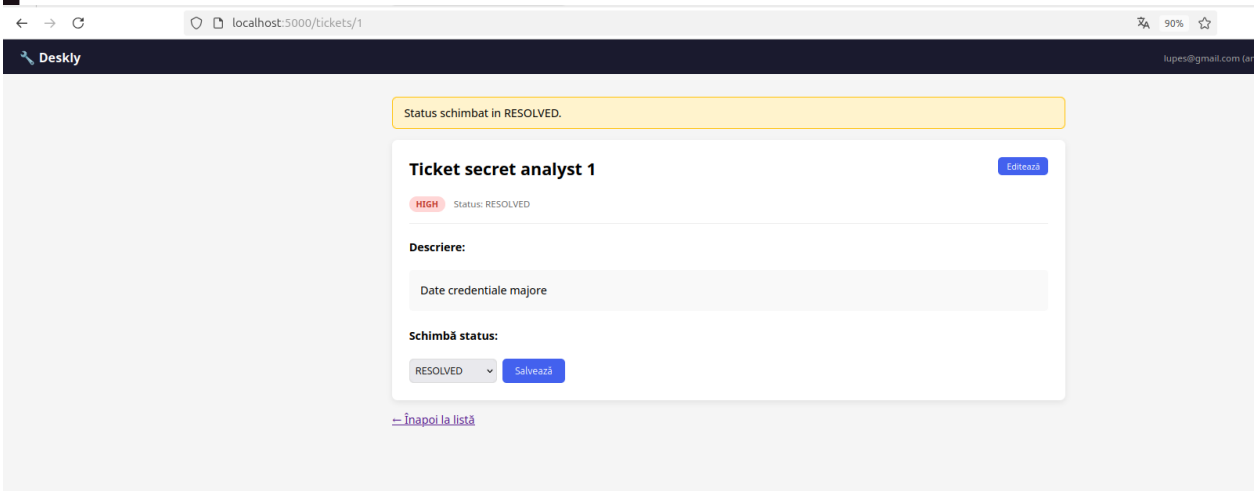
Endpoint-ul POST /tickets/{id}/status accepta request-uri fara token CSRF. Un atacator poate crea o pagina HTML externa care, atunci cand este accesata de un utilizator logat, trimite automat un request in numele sau.

Pagina malitioasa (csrf_attack.html):

```
<form id='csrf-form' action='http://localhost:5000/tickets/1/status' method='POST'>
  <input type='hidden' name='status' value='RESOLVED'>
</form>
<script>document.getElementById('csrf-form').submit();</script>
```

Rezultat: Dupa accesarea paginii malitioase si apasarea butonului, statusul tichetului s-a schimbat din OPEN in RESOLVED fara stirea utilizatorului.





Impact:

Atacatorul poate inchide/modifica tichete de incident, schimba statusuri, sau efectua orice actiune POST disponibila in aplicatie, in numele victimei.

5.5 PoC #5 - Error Handling & Info Disclosure

Descriere tehnica:

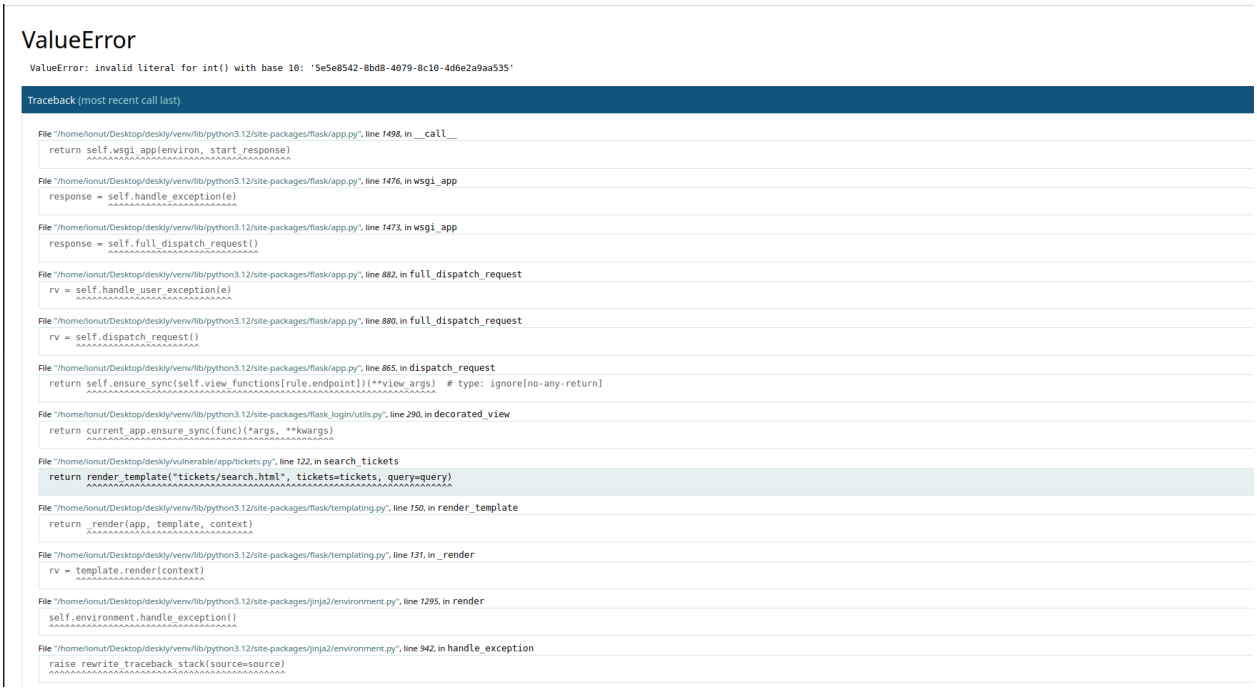
Aplicatia ruleaza cu debug=True si nu are un handler global de erori. Orice exceptie neprinda afiseaza stack trace-ul complet in browser, incluzand caile fisierelor de pe server, versiunile librariilor si detalii despre baza de date.

Codul vulnerabil:

```
except Exception as e:
    return f'<pre>Eroare DB: {str(e)}\nQuery: {sql}</pre>', 500
```

Informatii expuse in stack trace:

- Calea completa pe server: /home/ionut/Desktop/deskly/vulnerable/app/tickets.py
- Versiunea Python si Flask
- Query-ul SQL complet cu input-ul utilizatorului
- Eroarea SQLite cu detalii despre structura bazei de date



5.6 PoC #6 - Password Storage

Descriere tehnica:

In versiunea vulnerabila parolele sunt stocate in plain text in baza de date. Nu exista politica de parole si nu exista mecanism de lockout dupa incercari esuate.

Codul vulnerabil:

```
user = User(email=email, password=password, role=role)  # plain text!
```

Dovada - interogare directa DB:

```
sqlite3 deskly_vulnerable.db "SELECT email, password FROM users;"
```

Rezultat: lupes@gmail.com | 1234 (parola in clear text!)

Impact:

Daca baza de date este compromisa (prin SQLi sau acces direct), toate parolele utilizatorilor sunt imediat disponibile atacatorului, fara nicio procesare suplimentara.

6. Analiza impactului

Combinand vulnerabilitatile de mai sus, un atacator motivat ar putea realiza un atac complet in mai multi pasi:

5. Pasul 1: Creeaza un cont de analyst (inregistrarea e publica)
6. Pasul 2: Injecteaza payload XSS intr-un tichet vizibil managerului
7. Pasul 3: Fura cookie-ul sesiunii managerului prin XSS
8. Pasul 4: Cu sesiunea managerului, acceseaza audit log-ul si toate tichetele
9. Pasul 5: Folosind SQLi extrage toate parolele din baza de date

Rezultatul final: compromiterea completa a sistemului, acces la toate datele confidentiale si posibilitatea de a sterge sau modifica orice informatie.

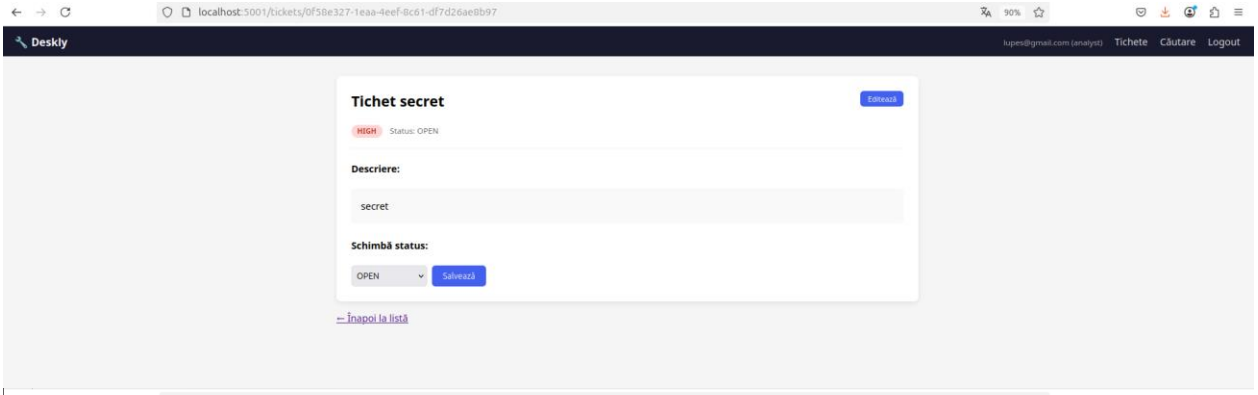
7. Implementarea fix-urilor

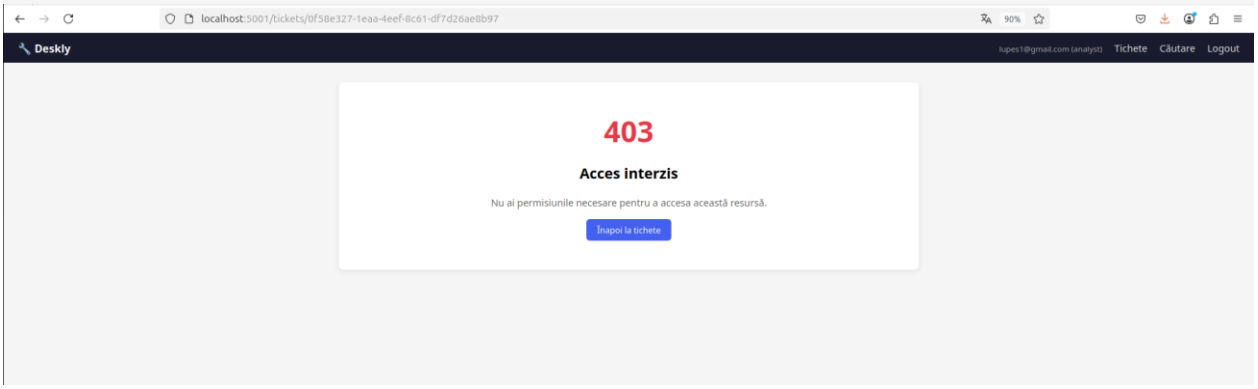
7.1 Fix IDOR - Autorizare server-side

Am implementat o functie de autorizare care verifica rolul si ownership-ul la fiecare endpoint sensibil:

```
def authorize_ticket(ticket, require_ownership=False):
    if current_user.role == 'manager':
        return  # manager are acces la tot
    if ticket.owner_id != current_user.id:
        log_action('UNAUTHORIZED_ACCESS', ...)
        abort(403)  # returneaza 403 Forbidden
```

In plus, am schimbat ID-urile tichetelor din integer secvential in UUID aleatoriu, eliminand posibilitatea de a ghici ID-uri valide.



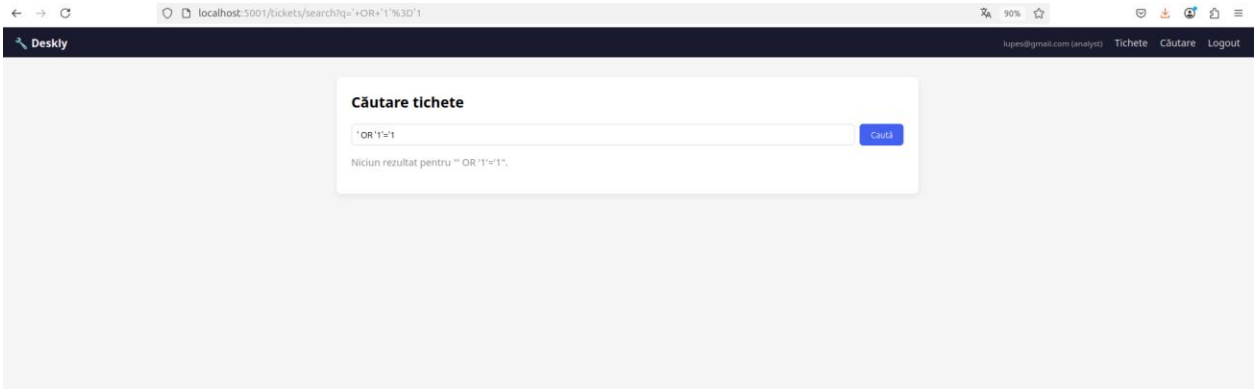


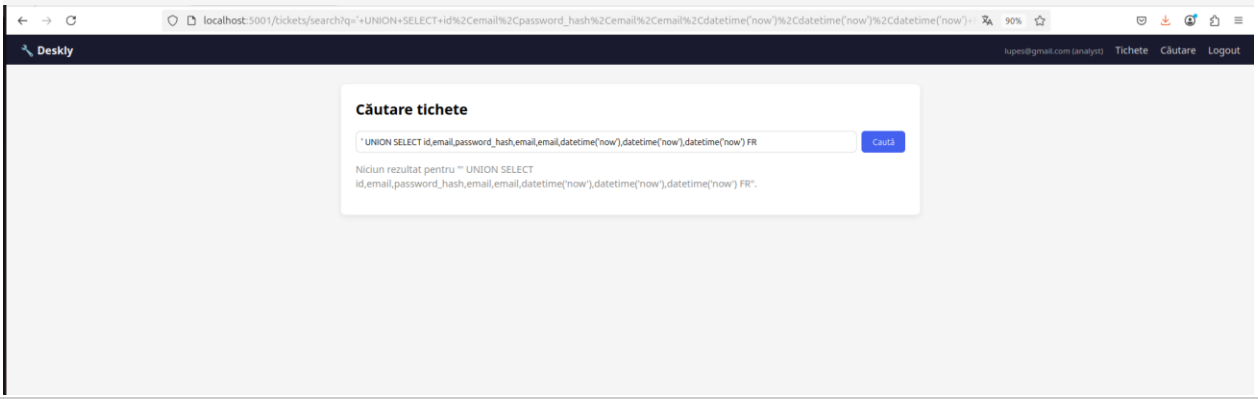
7.2 Fix SQL Injection - Query parametrizat

Am inlocuit concatenarea de string-uri cu ORM parametrizat SQLAlchemy:

```
search_term = f'%{query}%'
base_query = Ticket.query.filter(
    db.or_(
        Ticket.title.ilike(search_term),
        Ticket.description.ilike(search_term)
    )
).limit(50)
```

SQLAlchemy trateaza parametrii ca date simple, nu ca cod SQL, eliminand complet posibilitatea de injectie.





7.3 Fix XSS - Output escaping + sanitizare input

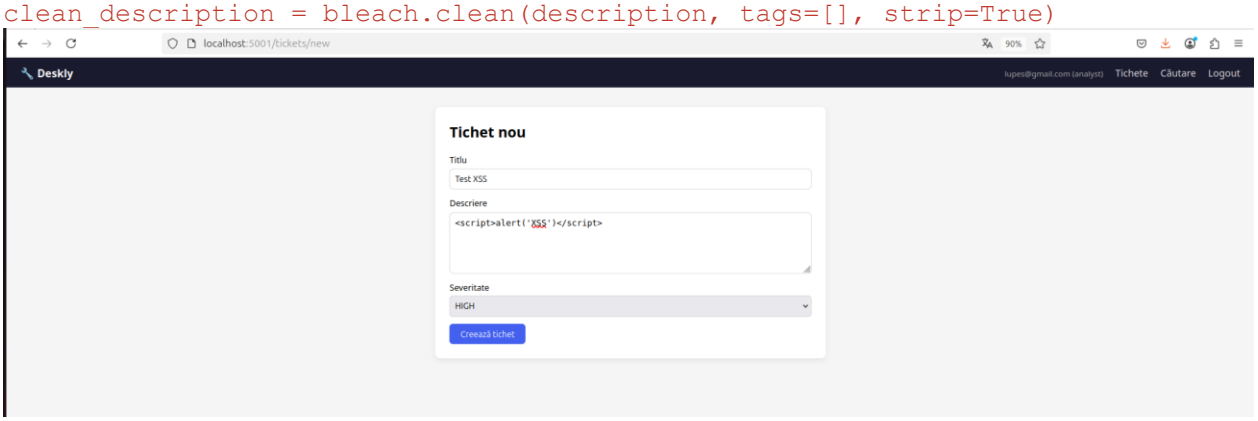
Am facut doua modificari importante:

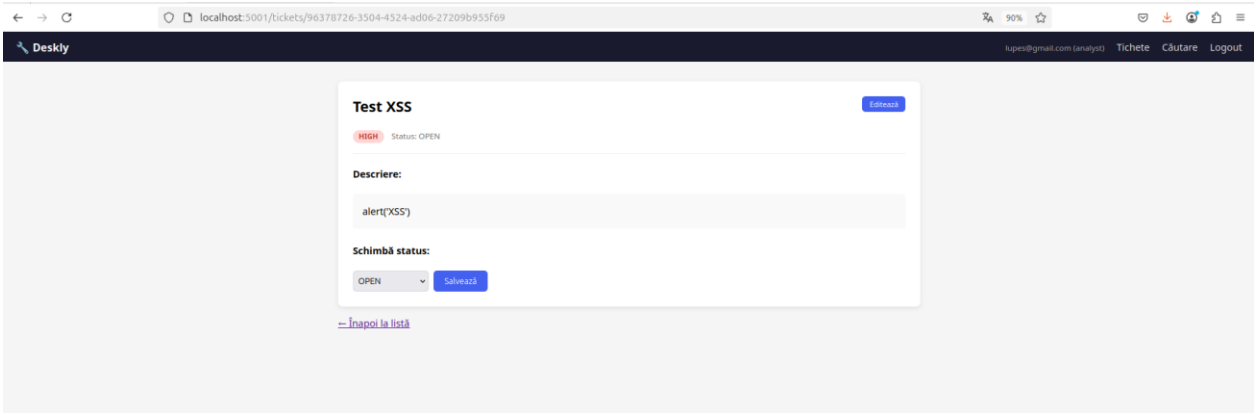
1. Am eliminat filtrul |safe din template - Jinja2 escapeaza automat implicit:

```
{{ ticket.description }}  <!-- fara |safe -->
```

2. Am adaugat sanitizare la input cu bleach:

```
clean_description = bleach.clean(description, tags=[], strip=True)
```





7.4 Fix CSRF - Flask-WTF protection

Am activat protectia CSRF globala prin Flask-WTF:

```
csrf = CSRFProtect()
csrf.init_app(app)  # protejeaza automat toate endpoint-urile POST
```

In fiecare formular am adaugat token-ul CSRF:

```
<input type='hidden' name='csrf_token' value='{{ csrf_token() }}'>
```

Am setat si cookie-ul cu SameSite=Lax pentru protectie suplimentara:

```
SESSION_COOKIE_SAMESITE = 'Lax'
```

In plus am setat debug=False in run.py pentru versiunea securizata.

7.5 Fix Password Storage - bcrypt + policy + lockout

Am implementat trei imbunatatiri:

1. Hash bcrypt cu cost factor 12:

```
pwd_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))
```

2. Politica de parole (minim 8 caractere, o litera mare, o cifra):

```
if len(password) < 8: return False
if not re.search(r'[A-Z]', password): return False
```

Parola trebuie să aibă minim 8 caractere.

Creare cont

Email

lupes@gmail.com

Parolă

••••

Rol

Analyst

Înregistrare

Ai deja cont? Loghează-te

Parola trebuie să aibă minim 8 caractere.

Creare cont

Email

lupes@gmail.com

Parolă

••••

Rol

Analyst

Înregistrare

Ai deja cont? Loghează-te

8. Re-test - Dovada ca atacurile nu mai functioneaza

#	Atac	Rezultat versiune FIXED
1	IDOR - acces ticket alt user	403 Forbidden + audit log
2	SQLi - OR 1=1	Niciun rezultat (query parametrizat)
3	SQLi - UNION SELECT	Niciun rezultat, nicio eroare DB
4	XSS Stored - script alert	Textul afisat ca text, fara executie
5	CSRF - schimbare status	400 Bad Request / Method Not Allowed
6	Parola slaba '1234'	Respinsa cu mesaj de eroare specific
7	Cookie din JavaScript	document.cookie returneaza '' (HttpOnly)

8.1 Teste automate

Am rulat 17 teste automate cu pytest care acopera toate vulnerabilitatile:

```
pytest tests/test_security.py -v
```

```
17 passed, 0 failed in 1.15s
```

Testele acopera: IDOR (4 teste), SQL Injection (3 teste), XSS (2 teste), Password Storage (4 teste), Authorization (4 teste).

ionut@ionut-lupes: ~/Desktop/deskly/tests

```
============================= no tests ran in 0.00s ==============================
ERROR: file or directory not found: tests/test_security.py

(venv) ionut@ionut-lupes:~/Desktop/deskly/tests$ pytest test_security.py -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0 -- /home/ionut/Desktop/deskly/fixed/venv/bin/python3.12
cachedir: .pytest_cache
rootdir: /home/ionut/Desktop/deskly/tests
collected 17 items

test_security.py::TestIDOR::test_analyst_cannot_view_other_users_ticket PASSED [  5%]
test_security.py::TestIDOR::test_analyst_cannot_edit_other_users_ticket PASSED [ 11%]
test_security.py::TestIDOR::test_manager_can_view_all_tickets PASSED     [ 17%]
test_security.py::TestIDOR::test_analyst_can_view_own_ticket PASSED      [ 23%]
test_security.py::TestSQLInjection::test_sql_injection_classic_or_1_1 PASSED [ 29%]
test_security.py::TestSQLInjection::test_sql_injection_union_select PASSED [ 35%]
test_security.py::TestSQLInjection::test_malicious_input_no_server_error PASSED [ 41%]
test_security.py::TestXSS::test_xss_payload_stored_as_text PASSED        [ 47%]
test_security.py::TestXSS::test_xss_payload_not_executable_in_response PASSED [ 52%]
test_security.py::TestPasswordStorage::test_password_stored_as_bcrypt_hash PASSED [ 58%]
test_security.py::TestPasswordStorage::test_different_users_same_password_different_hashes PASSED [ 64%]
test_security.py::TestPasswordStorage::test_bcrypt_verify_correct_password PASSED [ 70%]
test_security.py::TestPasswordStorage::test_bcrypt_reject_wrong_password PASSED [ 76%]
test_security.py::TestAuthorization::test_analyst_cannot_delete_ticket PASSED [ 82%]
test_security.py::TestAuthorization::test_analyst_cannot_access_audit_log PASSED [ 88%]
test_security.py::TestAuthorization::test_manager_can_delete_ticket PASSED [ 94%]
test_security.py::TestAuthorization::test_unauthenticated_user_redirected PASSED [100%]

============================= warnings summary ===============================
test_security.py: 63 warnings
  /home/ionut/Desktop/deskly/fixed/venv/lib/python3.12/site-packages/sqlalchemy/sql/schema.py:3585: DeprecationWarning: datet
ime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent da
tetimes in UTC: datetime.datetime.now(datetime.UTC).
    return util.wrap_callable(lambda ctx: fn(), fn)  # type: ignore

test_security.py: 12 warnings
  /home/ionut/Desktop/deskly/tests/../fixed/app/auth.py:95: DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(da
tetime.UTC).
    user.last_login_at = datetime.utcnow()
```

9. Audit Log si trasabilitate

Aplicatia inregistreaza automat toate evenimentele importante in tabela audit_logs. Fiecare intrare contine: utilizatorul, actiunea, resursa afectata, IP-ul si timestamp-ul.

Evenimente inregistrate:

- REGISTER, LOGIN_SUCCESS, LOGIN_FAILED, LOGOUT
- TICKET_CREATE, TICKET_VIEW, TICKET_UPDATE, TICKET_DELETE
- TICKET_STATUS_CHANGE, SEARCH
- UNAUTHORIZED_ACCESS (bonus - tentative IDOR loggate)

In versiunea securizata, tentativele de acces neautorizat (IDOR) sunt logate cu actiunea UNAUTHORIZED_ACCESS, permitand detectarea atacatorilor chiar daca atacul a esuat.

Audit log-ul este accesibil la /audit si este protejat - doar utilizatorii cu rol Manager il pot vizualiza.

10. Concluzii

Acest proiect a fost destul de instructiv. Mi-am dat seama ca vulnerabilitatile de securitate nu sunt neaparat complexe - de multe ori sunt lucruri simple pe care le uiti sau le ignori cand esti grabit sa livrezi ceva.

Lucruri pe care le-am invatat:

- Niciodata nu afisa erori tehnice utilizatorului - logheaza-le, dar arata mesaje generice
- Input-ul utilizatorului e intotdeauna suspect - valideaza si sanitizeaza orice
- Nu te baza pe client pentru securitate - verifica totul server-side
- Parolele in plain text sunt o catastrofa - bcrypt e standard si usor de implementat
- CSRF si XSS sunt usor de prevenit daca folosesti framework-ul corect

Cel mai interesant mi s-a parut atacul XSS - nu ma gandeam ca un simplu tag <script> intr-un camp de text poate duce la furtul sesiunii unui administrator. Demonstratia cu cookie-ul din popup a fost destul de convingatoare.

Daca as fi facut proiectul fara cerinta de securitate, probabil as fi uitat de jumatate din aceste aspecte si as fi livrat o aplicatie cu toate vulnerabilitatile din versiunea MVP.